

Assignment Overview

- **Objectives:** You are the Chief Data Scientist of the night market vendor alliance, which wants a soundtrack for every zone of the market: calm music near the tea stalls, upbeat tracks around the games, and something loud for the late night crowd. Instead of relying on genre tags, the alliance asks you to discover natural *moods* directly from the audio features of real Spotify tracks, and to flag tracks that fit no mood at all before they end up on a playlist. Your job is to work through **partitional**, **hierarchical**, and **density-based clustering** by hand, implement ***k*-means** on real data, and build and evaluate **outlier detectors** from the proximity-based, clustering-based, and isolation families.
- **Constraints:**
 1. You must implement ***k*-means** (Part II) and the ***k*NN distance** and **LOF** detectors (Part III) **from scratch** in Python using `numpy`. You may use `pandas`, `matplotlib`, and the Python standard library.
 2. In Part III, you may use `sklearn.cluster.DBSCAN` and `sklearn.ensemble.IsolationForest` as reference detectors.
 3. Apart from these reference detectors, `scikit-learn` may be used **only** to verify your results and to compute evaluation metrics (silhouette score, adjusted Rand index, ROC AUC, precision, recall, and F1), never as your submitted *k*-means or LOF.
 4. All code must run on Google Colab (free tier CPU is sufficient). We recommend:
 - Use **SEPARATE** notebooks for each part.
 - Do not clear your notebook outputs before submitting. Markers need to see the **ACTUAL** results of your code, not just the code itself.
 - Fix all random seeds to 0 unless a task asks for several seeds, so that your results are reproducible.
- **General Instructions:** The assignment consists of three compulsory components:
 1. **Part I:** Clustering and PCA by Hand (20 Points)
 2. **Part II:** *k*-means from Scratch (35 Points)
 3. **Part III:** Outlier Detection (45 Points)
- **Submission Guidelines:** Submit your work through the e3 submission system, including the following files:
 1. `A1_[ID].zip`: Your complete codebase, consisting of **SEPARATE** notebooks for each part.
 2. `A1_[ID].pdf`: A PDF report containing your handwritten or typeset solutions to Part I, code snippets, all requested tables and plots, and your written analysis. Include a link to your Google Colab notebooks (make sure they are viewable).
- **Late Submission Policy:**
 1. 25% penalty: From 15 minutes to 1 day late
 2. 50% penalty: From 1 day to 2 days late
 3. 100% penalty: More than 2 days late

Part I: Clustering and PCA by Hand [20 marks]

You will run each clustering method by hand on small examples and see why PCA must centre the data.

1. Tasks.

- (a) **Why Centre? (4 points):** PCA computes the eigenvectors of the covariance matrix of the *centred* data. Consider

$$M = \begin{bmatrix} 4 & 2 \\ 2 & 4 \\ 3 & 3 \\ 3 & 3 \end{bmatrix}.$$

- Compute $M^T M$ and its eigenvalues and unit eigenvectors by hand.
 - Centre M by subtracting the column means to obtain $\tilde{M}$. Compute the covariance matrix $C = \frac{1}{4} \tilde{M}^T \tilde{M}$, together with its eigenvalues and unit eigenvectors.
 - The leading eigenvector changes after centring. Explain why, and state which direction PCA should report as the first principal component.
- (b) **k -means by Hand (6 points):** Seven stalls sit along a street at positions $\{1, 2, 3, 8, 9, 10, 25\}$. Run k -means with $k = 2$ and squared Euclidean cost $\phi = \sum_x \min_c (x - c)^2$.
- **Initialisation A:** centroids $\{1, 25\}$. **Initialisation B:** centroids $\{1, 2\}$. For each, list the assignments, centroids, and cost ϕ at every iteration until convergence.
 - Which initialisation reaches the lower final cost? Is the other result a global optimum? Explain why k -means can converge to different solutions.
 - **Farthest-first** initialisation picks a first centroid and then repeatedly adds the point farthest from all chosen centroids. Starting from position 1, which centroids does it choose for $k = 2$ and for $k = 3$? Comment on how this strategy reacts to the stall at 25.
- (c) **Hierarchical Clustering by Hand (5 points):** Along another street, five food carts stand at positions $\{0, 2, 3, 7, 12\}$; the distance between carts at x and y is $|x - y|$.
- Run agglomerative clustering with **single link (MIN)** and with **complete link (MAX)**. For each, list the clusters merged at every step with the merge height, and draw the dendrogram.
 - Cut each dendrogram into two clusters. Where does the cart at 7 end up in each case? Use this example to explain the chaining effect of single link.
- (d) **DBSCAN by Hand (5 points):** Return to the seven stalls in (b). Use $\varepsilon = 1$ and $MinPts = 3$, where the ε -neighbourhood of a point includes the point itself.
- Classify every stall as a core, border, or noise point, and list the resulting clusters.
 - Compare this result with the two k -means solutions in (b). Which method handles the stall at 25 more sensibly, and why?

Part II: k -means from Scratch [35 marks]

The alliance has shortlisted four genres that could suit different zones of the market and now wants concrete playlists. You will prepare the data, implement k -means, study its sensitivity to initialisation, and test whether the moods it finds are real.

1. **The Dataset.** We use the **Spotify Tracks Dataset**¹, which contains 114,000 tracks from 114 genres (1,000 tracks per genre), each described by metadata and audio features computed by Spotify.

- **Features:** the nine audio features danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, and tempo.
- **Labels (not used for fitting):** `track_genre`. Keep `track_name` and `artists` for inspecting individual tracks.
- **Loading:** download `dataset.csv` from the dataset page, or read it directly in Colab with

```
pandas.read_csv('hf://datasets/maharshipandya/spotify-tracks-dataset/dataset.csv')
```

- **Track map:** whenever a task asks you to show results on the *track map*, draw a scatter plot of the standardised acousticness (horizontal axis) against the standardised energy (vertical axis).

2. **Setup.** For a set of centroids $\mathcal{C}$, define the cost $\phi = \sum_x \min_{c \in \mathcal{C}} \|x - c\|_2^2$ (the SSE). Let $\phi(0)$ denote the cost under the initial centroids before any update, and $\phi(t)$ the cost after the t -th iteration. For seeded initialisations, draw the first point as `numpy.random.default_rng(seed).integers(n)`.

3. **Tasks.**

(a) **Data Preparation (5 points):**

- Keep the tracks whose `track_genre` is classical, hip-hop, mandopop, or metal. Remove rows with a duplicated `track_id`, and then rows whose nine audio features exactly duplicate an earlier row (the same recording released on several albums).
- Standardise each of the nine features to zero mean and unit variance using the population standard deviation (`ddof=0`). Call the result $Z \in \mathbb{R}^{n \times 9}$. Use Z in all later tasks.
- Report n and the number of tracks per genre. Report the skewness of each feature, and explain in two sentences how strongly skewed features can affect distance based methods.
- Draw the track map coloured by genre. Which genre stands apart, and which genres overlap?
- **Sanity check:** you should keep $n = 3,408$ tracks.

(b) **Implementation (8 points):** Write a function `kmeans(X, CO, max_iter=20)` that returns the final assignments, the final centroids, and the cost history $\phi(0), \dots, \phi(\text{max_iter})$.

- Assign each point to its nearest centroid, breaking ties by the smallest centroid index, and update each centroid as the mean of its assigned points. If a cluster becomes empty, keep its previous centroid.
- Vectorise the distance computation with `numpy`; explicit Python loops over data points are not allowed.
- Verify on Z that your final SSE matches `sklearn.cluster.KMeans` started from the same initial centroids.

(c) **Initialisation Strategies (12 points):** With $k = 4$, compare three initialisations, each run with seeds $0, \dots, 19$:

- **Random:** k distinct data points chosen uniformly at random.
- **Farthest-first:** a random first point, then repeatedly the point with the largest distance to its nearest chosen centroid.
- **k -means++:** a random first point, then each next point sampled with probability proportional to its squared distance to the nearest chosen centroid.

For each strategy, plot $\phi(t)$ against t for seed 0, and report the median $\phi(0)$ and the minimum, median, and maximum $\phi(20)$ over the 20 seeds. Which strategy starts best and which ends best?

¹Spotify Tracks Dataset. Hugging Face, 2022. <https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset>. The data were collected through the Spotify Web API; see the dataset card for details and terms of use.

- List the four tracks that farthest-first selects with seed 0 (name, artist, genre, and their most extreme features). Why are these tracks chosen, and what does this reveal about the sensitivity of farthest-first to outliers?
 - **Sanity check:** the best $\phi(20)$ over all runs should be close to 1.58×10^4 .
- (d) **Choosing k and Cluster Validity (10 points):** For $k = 2, \dots, 8$, run k -means with k -means++ initialisation over seeds $0, \dots, 9$ and keep the run with the lowest $\phi(20)$.
- Plot the best SSE (elbow plot), the mean silhouette coefficient, and the adjusted Rand index (ARI) against genre as functions of k .
 - Which k does each criterion favour? Explain why an internal criterion (silhouette) and an external one (ARI against genre) can disagree, and recommend a number of playlists to the alliance. Show the best $k = 4$ clusters on the track map.
 - **Is the structure real?** Build 10 null datasets by independently permuting each column of Z , which keeps every feature's distribution but destroys the dependence between features. Run the same procedure on each and plot the mean null SSE alongside the real SSE for every k . What does the gap tell you?

Part III: Outlier Detection [45 marks]

Part II hinted that some tracks fit no mood: spoken recordings, live bootlegs, or pieces so quiet they are barely audible. You will now build outlier detectors from three families, check whether they agree, and evaluate them against anomalies whose identity you control.

1. **Definitions.** Let $d(\cdot, \cdot)$ be the Euclidean distance, $N_k(o)$ the set of the k nearest neighbours of o (excluding o itself), and $\text{dist}_k(o)$ the distance from o to its k -th nearest neighbour. The Local Outlier Factor is defined through

$$\text{reach}_k(o \leftarrow o') = \max\{\text{dist}_k(o'), d(o, o')\}, \quad \text{lrd}_k(o) = \left(\frac{1}{|N_k(o)|} \sum_{o' \in N_k(o)} \text{reach}_k(o \leftarrow o') \right)^{-1},$$
$$\text{LOF}_k(o) = \frac{1}{|N_k(o)|} \sum_{o' \in N_k(o)} \frac{\text{lrd}_k(o')}{\text{lrd}_k(o)}.$$

$\text{LOF}_k(o) \approx 1$ means o is as dense as its neighbours; $\text{LOF}_k(o) > 1$ means it is sparser, suggesting an outlier.

Memory tip: with $n \approx 3,400$, broadcasting an $n \times n \times 9$ array needs almost 1 GB; compute pairwise distances via $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b$ instead.

2. Tasks.

- (a) **Warm-up by Hand (7 points):** Return to the seven stalls $\{1, 2, 3, 8, 9, 10, 25\}$ from Part I.

- **z -score:** Estimate μ and σ by maximum likelihood and compute the z -score of the stall at 25. Is it flagged by the rule $|z| > 3$? Explain why the outlier itself makes it harder to detect (the masking effect).
- **LOF:** With $k = 2$, compute dist_2 , N_2 , lrd_2 , and LOF_2 for every stall. Which stall is the top outlier? Some stalls inside the two groups obtain $\text{LOF}_2 > 1$; explain why, and what this implies for choosing a threshold.

- (b) **Proximity-Based Approaches (14 points):**

- Write `knn_score(X, k)`, returning $\text{dist}_k(o)$ for every point, and `lof(X, k)`, implementing the definitions above with vectorised numpy operations.
- Verify for $k = 20$ that your LOF agrees to within 10^{-8} with scikit-learn's `LocalOutlierFactor`, whose attribute `negative_outlier_factor_` stores the negated LOF. The duplicate removal in Part II (a) matters here: explain why exact duplicates break LOF.
- Report the number of tracks with $\text{LOF}_{20} > 1.5$ and list the five tracks with the largest LOF (name, artist, genre). Show the k NN and LOF scores on the track map as colour intensity.
- **Sanity check:** roughly 38 tracks should exceed $\text{LOF}_{20} = 1.5$, and the largest value should be about 2.6.

- (c) **Clustering-Based and Isolation Approaches (10 points):**

- Using the best $k = 4$ solution of Part II (d), score each track by $\text{dist}(o, c_o) / \overline{\text{dist}}(c_o)$, its distance to its centroid divided by the average distance of that cluster's members to the centroid.
- Run `sklearn.cluster.DBSCAN` with $\varepsilon = 2.0$ and `min_samples=18` (twice the dimensionality; the neighbourhood includes the point itself, as in Part I (d)), and take its m noise points as a binary detector. **Sanity check:** you should obtain one cluster and $m = 55$ noise points.
- Fit an `IsolationForest` with 200 trees and `random_state=0`; its negated `score_samples` serves as the outlier score.
- On the track map, mark the m highest scoring tracks of each detector. Which detectors agree most with the DBSCAN noise points, and why? Listen to (or look up) two flagged tracks: are they errors, or genuinely unusual music?

- (d) **Evaluation with Injected Anomalies (14 points):** Real data have no outlier labels, so you will plant 20 anomalies of two kinds and append each set separately to Z (with labels 1 for planted points and 0 otherwise):

- **Global:** with `rng = numpy.random.default_rng(0)`, draw $U = \text{rng.normal}(\text{size}=(20, 9))$, normalise each row to unit length, and scale row i by a radius drawn from `rng.uniform(4, 6, (20, 1))`.

- **Correlation-breaking:** with `rng = numpy.random.default_rng(1)`, fill the planted points column by column: for $j = 0, \dots, 8$ in order, set column j to `Z[rng.integers(0, n, 20), j]`. Every value is realistic on its own, but the combination is not, like a track that is simultaneously quiet, danceable, and fully instrumental.

Re-run every scoring detector (k NN distance with $k = 20$, LOF_{20} , the clustering-based score, and Isolation Forest) on each augmented set, refitting k -means with $k = 4$ for the clustering-based score. Report a table of ROC AUC and precision among the top 20 scores for both anomaly types, plot the ROC curves, and report the precision, recall, and F1 of the DBSCAN noise detector.

- Why would accuracy be a misleading metric here?
- Which detectors succeed on global anomalies, and which remain useful on correlation-breaking ones? Relate your answer to the assumptions each family makes about normal data.
- Recommend one detector to the alliance for screening new tracks before they enter a playlist, and justify it in three to four sentences.
- **Sanity check:** every scoring detector should reach a ROC AUC above 0.95 on global anomalies, and DBSCAN should flag all 20 of them. On correlation-breaking anomalies, LOF should perform best at roughly 0.9, whereas Isolation Forest should drop to roughly 0.7.